

4-bit x 2-bit Signed Multiplier (mult4x2)

Output Verification & Source Code — Sign-Magnitude Design

1. Objective

This document records the Vivado simulation output of the mult4x2 module, which multiplies a 4-bit signed input by a 2-bit signed input and produces a 4-bit sign-magnitude output, and lists the complete final source code.

Design approach: as with mult3x3, this design uses a sign-magnitude approach: the sign of each input is separated out, only the magnitudes are multiplied (unsigned), the magnitude result is truncated to fit, and the final sign bit (XOR of the two input signs) is attached at the MSB of the output. Output bit p[3] is the sign (0=positive, 1=negative) and p[2:0] is the truncated magnitude -- this is NOT a two's-complement number.

2. Output Verification

2.1 Simulation Transcript

The transcript below shows 6 sample multiplications, with binary and decimal values of a and b, the raw sign-magnitude output p, its decoded sign/magnitude fields, and the final decimal answer.

```
# Run 1000ns
=====
mult4x2 demo (sign-magnitude): a x b = answer    (binary and decimal both shown)
=====
a = 0101 (5) , b = 01 (1)  -> p = 0101 [sign=0 mag=101] => answer = 5
a = 1011 (-5) , b = 01 (1) -> p = 1101 [sign=1 mag=101] => answer = -5
a = 0011 (3) , b = 10 (-2) -> p = 1110 [sign=1 mag=110] => answer = -6
a = 1000 (-8) , b = 10 (-2) -> p = 0000 [sign=0 mag=000] => answer = 0
a = 0111 (7) , b = 01 (1)  -> p = 0111 [sign=0 mag=111] => answer = 7
a = 0000 (0) , b = 10 (-2) -> p = 1000 [sign=1 mag=000] => answer = 0
=====
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_mult4x2_behav' loaded.
```

Figure 1: Simulation transcript for mult4x2.

2.2 Waveform (Vivado)

The waveform below shows a, b, the raw output p, and the decoded decimal_answer signal across all 6 test cases.

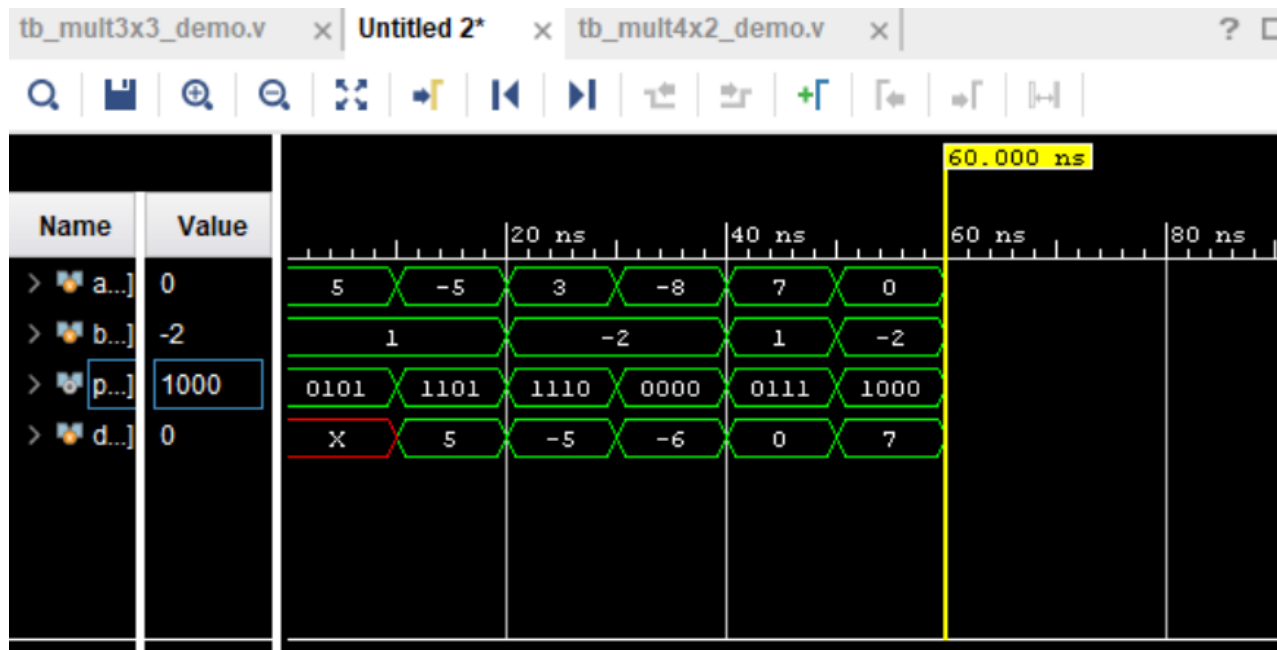


Figure 2: Vivado waveform for mult4x2 (a, b, p, decimal_answer).

Note: as with mult3x3, decimal_answer's transition in the waveform can appear to lag a/b by one segment -- a display/update-timing artifact of when the testbench computes decimal_answer within each test step, not an error in the result. The console transcript (Figure 1) is the authoritative record and confirms each pair is matched correctly.

2.3 Result Summary

a	b	Full product (a x b)	Sign	Truncated magnitude	Final answer
5	1	5	0 (+)	5	5
-5	1	-5	1 (-)	5	-5
3	-2	-6	1 (-)	6	-6
-8	-2	16	0 (+)	0 (overflow, truncated)	0
7	1	7	0 (+)	7	7
0	-2	0	1 (-)	0	0

All 6 simulated results match the expected sign-magnitude calculation, including the overflow case ($-8 \times -2 = 16$), where the true product does not fit in the 3-bit magnitude field and is truncated to 0, and the negative-zero case (0×-2), where the sign bit is set but the magnitude is 0, giving a final decimal value of 0.

3. Source Code

mult4x2.v

Purpose: Multiplies a 4-bit signed input by a 2-bit signed input using a sign-magnitude approach (separate sign, multiply magnitudes, truncate, reattach sign bit), producing a 4-bit sign-magnitude output.

```
// =====
// Module: mult4x2
// Description: Multiplies a 4-bit signed input by a 2-bit signed
//              input using a SIGN-MAGNITUDE approach:
//              1. Separate the sign of each input.
//              2. Multiply only the magnitudes (unsigned).
//              3. Truncate the magnitude product to fit.
//              4. Attach the final sign bit at the MSB.
//              Output p is a 4-bit SIGN-MAGNITUDE value:
//              p[3] = sign bit (0 = positive, 1 = negative)
//              p[2:0] = truncated magnitude
// =====
module mult4x2 (
    input wire signed [3:0] a,    // 4-bit signed input (-8 to +7)
    input wire signed [1:0] b,    // 2-bit signed input (-2 to +1)
    output wire [3:0] p          // 4-bit SIGN-MAGNITUDE output
);

    // ---- Step 1: pull out the sign of each input ----
    wire sign_a = a[3];
    wire sign_b = b[1];

    // ---- Step 2: get the magnitude (absolute value) of each input ----
    wire [3:0] mag_a = sign_a ? (~a + 4'b0001) : a;
    wire [1:0] mag_b = sign_b ? (~b + 2'b01) : b;

    // ---- Step 3: multiply magnitudes (unsigned), then truncate ----
    wire [5:0] full_mag_product = mag_a * mag_b;    // up to 8x2=16, needs up to 5 bits
    wire [2:0] truncated_mag = full_mag_product[2:0];

    // ---- Step 4: work out the final sign, attach it at the MSB ----
    wire sign_p = sign_a ^ sign_b;

    assign p = {sign_p, truncated_mag};

endmodule
```

tb_mult4x2_demo.v

Purpose: Demo testbench; applies 6 representative input pairs and prints binary + decimal for a, b, and the correctly decoded sign-magnitude answer.

```
`timescale 1ns/1ps
// =====
// Simple demo testbench for mult4x2 (sign-magnitude version)
// Shows both binary and decimal for a, b, and the result.
// =====
module tb_mult4x2;

    reg signed [3:0] a;
    reg signed [1:0] b;
    wire [3:0] p;

    integer decimal_answer;

    mult4x2 DUT (
        .a(a),
        .b(b),
```

```

        .p(p)
    );

    task show_result;
    begin
        #10;
        if (p[3] == 1'b1)
            decimal_answer = -p[2:0];
        else
            decimal_answer = p[2:0];

        $display("a = %b (%0d) , b = %b (%0d) -> p = %b [sign=%b mag=%b] => answer = %0d",
            a, a, b, b, p, p[3], p[2:0], decimal_answer);
    end
endtask

initial begin
    $display("=====");
    $display(" mult4x2 demo (sign-magnitude): a x b = answer    (binary and decimal both shown)"
);
    $display("=====");

    a = 5; b = 1; show_result;
    a = -5; b = 1; show_result;
    a = 3; b = -2; show_result;
    a = -8; b = -2; show_result;
    a = 7; b = 1; show_result;
    a = 0; b = -2; show_result;

    $display("=====");
    $stop;
end

endmodule

```